

Google Antigravity Versión 2.0

¿Está “Gemini Code Assist” disponible como extensión en la versión 2.0 de “Google Antigravity”?

No, Gemini Code Assist no está disponible como una extensión dentro de **Google Antigravity 2.0**. De hecho, Google ha rediseñado por completo su ecosistema y la actualización 2.0 cambia radicalmente las cosas: [1, 2, 3]

- **Antigravity 2.0 ya no es un IDE tradicional:** Se ha transformado en una aplicación de escritorio independiente (“*agent-first*”) enfocada puramente en la orquestación y ejecución de agentes autónomos de IA en paralelo. No cuenta con un editor de código convencional de manera nativa ni un mercado de extensiones para asistentes del tipo autocompletar. [2, 4, 5, 6, 7]
- **La unificación de Google:** Google ha integrado y absorbido las funciones de Gemini Code Assist individuales y Gemini CLI dentro de la propia plataforma de **Antigravity 2.0** y **Antigravity CLI**. [1, 8]
- **El fin del soporte individual:** Las extensiones de Gemini Code Assist para usuarios individuales dejarán de operar. Si eres un usuario gratuito o independiente, Google te migrará de forma directa a usar el plan individual de Antigravity. [1, 8, 9]

¿Cómo conviven estas herramientas actualmente?

Para entender el nuevo ecosistema, Google dividió su software en tres variantes principales: [2]

1. **Google Antigravity 2.0 (App de escritorio)**: Es el centro de comando para lanzar instrucciones complejas (“Modo Agente”). Delegas la tarea, la IA genera un plan, abre subagentes y edita archivos en segundo plano sin que tengas que programar línea por línea. [10, 11, 12, 13]
2. **Antigravity ID (El Editor)**: Si extrañas la interfaz clásica parecida a VS Code (donde tenías el código a la izquierda y el chat a la derecha), Google mantiene por separado **Antigravity ID**. En esta versión sí puedes usar extensiones tradicionales y un flujo de trabajo típico de desarrollo. [2, 6, 14]
3. **Antigravity CLI**: El sustituto oficial de terminal basado en Go que reemplaza a Gemini CLI por completo. [2, 6]

Nota para empresas: Si en tu organización pagan una licencia de **Gemini Code Assist Standard o Enterprise**, esas extensiones en IDEs externos (como VS Code tradicional o JetBrains) seguirán funcionando con normalidad y no se verán afectadas por este cambio. [1, 15]

Si utilizabas la versión gratuita de Gemini Code Assist, te recomiendo descargar directamente la suite de **Antigravity 2.0** para empezar a experimentar con el desarrollo basado en agentes autónomos. ¿Te gustaría que te explique cómo configurar tus proyectos en esta nueva interfaz de agentes o prefieres ver cómo instalar **Antigravity ID** para conservar un editor tradicional? [2, 9]

- [1] <https://developers.googleblog.com/an-important-update-transitioning-gemini->
- [2] <https://www.youtube.com/watch?v=RZ1ZSRuShn4>
- [3] <https://www.youtube.com/watch?v=KR4PDUnft0g>
- [4] https://www.youtube.com/watch?v=m9F05aM_1gw&v1=es-419
- [5] <https://ecosistemastartup.com/google-antigravity-2-0-crea-agentes-ia-con-1-in>
- [6] <https://www.youtube.com/shorts/UukjXOMLRnM>
- [7] https://www.reddit.com/r/google_antigravity/comments/1thxv5g/introducing_google_antigravity_20/?tl=es-419
- [8] <https://www.forgenex.com/public/blog/antigravity-2-0-google-unifica-sus-herram>
- [9] <https://developers.google.com/gemini-code-assist/resources/faqs>
- [10] https://www.youtube.com/watch?v=_Ed5y0QfvWU
- [11] <https://antigravityide.org/>
- [12] <https://antigravityide.org/>
- [13] <https://www.youtube.com/watch?v=-0Irz8G0PEE>
- [14] <https://www.youtube.com/watch?v=jrhJC6cpsmI&t=82>
- [15] <https://fazt.dev/contenido/google-antigravity-2-0-gemini-3-5-flash>

En la versión clásica parecida a VS Code, ¿está disponible “Gemini Code Assist” entre las extensiones tradicionales?

Sí, pero ya no es necesario instalarla. En la interfaz clásica basada en VS Code de Antigravity (el editor integrado), **las funciones de Gemini Code Assist vienen preinstaladas de forma nativa como el motor de IA principal del sistema.** [1, 2, 3]

Si de todos modos buscas la extensión tradicional de “Gemini Code Assist” (o la antigua de Google Cloud Code) dentro de su menú de extensiones, debes tener en cuenta los siguientes puntos técnicos clave:

- **Viene integrada en el núcleo:** Al abrir el editor de Antigravity por primera vez, verás que la barra lateral de chat de IA, el autocompletado en línea y las predicciones ya están impulsados directamente por Gemini. No tienes que ir a la tienda a buscarla. [1, 4]
- **El mercado de extensiones por defecto es Open VSX:** Antigravity es una bifurcación de código abierto de VS Code, por lo que su catálogo nativo de extensiones es Open VSX. Dado que la extensión tradicional de Gemini Code Assist es propiedad exclusiva de Google para el IDE de Microsoft, **no aparece en la búsqueda inicial.** [5, 6]
- **Cómo usar extensiones externas:** Si por algún motivo necesitas forzar una versión específica o instalar extensiones exclusivas de Microsoft, los desarrolladores suelen modificar la configuración de Antigravity para apuntar al Marketplace oficial de VS Code, o instalan manualmente los archivos descargados en formato `.vsix`.

En resumen: puedes programar con el flujo tradicional de VS Code en Antigravity y tendrás todo el poder de Gemini Code Assist activo desde el primer minuto sin descargar ningún complemento adicional. [1, 3, 7]

¿Te gustaría saber cómo **importar directamente tu configuración, temas y atajos desde tu VS Code actual** hacia Antigravity para no perder tu entorno personalizado? [2]

[1] <https://www.instagram.com/reel/DRNfWl8D31i/?hl=es>

[2] <https://codelabs.developers.google.com/getting-started-google-antigravity?hl=es-419>

[3] https://www.reddit.com/r/google_antigravity/comments/1sbegkw/vs_code_vs_googles_antigravity/?tl=es-419

[4] <https://developers.google.com/gemini-code-assist/docs/write-code-gemini>

[5] https://www.youtube.com/watch?v=JWo_SaV95dM

[6] <https://translate.google.com/translate?u=https://www.xda-developers.com/googles-antigravity-beats-vs-code/&hl=es&sl=en&tl=es&client=>

sge

[7] <https://translate.google.com/translate?u=https://jimmysong.io/blog/antigravity-vscode-style-ide/&hl=es&sl=en&tl=es&client=sge>
necesito saber como importar mi configuración, temas y atajos que tengo en mi “Google Antigravity” viejo al nuevo “Google Antigravity”

Para migrar tu configuración, temas y atajos desde la versión vieja de Google Antigravity a la nueva, debes tener en cuenta que el salto a la versión 2.0 modificó por completo las carpetas del sistema. La actualización automática suele reescribir los directorios y “esconder” tus preferencias anteriores en una carpeta de respaldo. [1, 2]

Dado que tus temas y atajos de teclado tradicionales pertenecen al entorno de desarrollo clásico, el destino correcto para restaurarlos es la aplicación **Antigravity IDE** (y no la app de escritorio 2.0 basada exclusivamente en agentes). [3, 4]

Aplica el siguiente procedimiento manual según tu sistema operativo para transferir tus datos de forma inmediata: [5]

En Windows

1. Presiona las teclas **WIN + R**, escribe **%APPDATA%** en el cuadro de ejecución y presiona Enter.
2. Ubica la carpeta llamada **Antigravity** (que contiene tus datos viejos) y cópiala temporalmente a tu Escritorio como respaldo.
3. Instala y abre por primera vez el nuevo **Antigravity IDE** para que genere sus nuevas rutas; luego ciérralo por completo.
4. Vuelve a **%APPDATA%**, abre la nueva carpeta llamada **Antigravity IDE** y pega allí todo el contenido que guardaste en el paso 2 (sobrescribiendo si es necesario). [4, 6, 7]

Si prefieres hacerlo rápido por consola, abre la terminal CMD y ejecuta estos comandos: [5]

```
xcopy "%APPDATA%\Antigravity" "%APPDATA%\Antigravity IDE" /E /I /H /Y  
xcopy "%USERPROFILE%\antigravity" "%USERPROFILE%\antigravity-ide" /E /I /H /Y
```

En macOS

Abre la Terminal y ejecuta los siguientes comandos para clonar tu configuración global y tus extensiones hacia el nuevo entorno: [5]

```
cp -rf ~/.antigravity ~/.antigravity-ide
cp -rf ~/Library/Application\ Support/Antigravity ~/Library/Application\ Support
```

En Linux

Abre tu Terminal y copia los directorios ocultos ejecutando: [5]

```
cp -rf ~/.antigravity ~/.antigravity-ide
cp -rf ~/.config/Antigravity ~/.config/Antigravity\ IDE
```

Migración de comandos para la Terminal (CLI)

Si además utilizabas atajos de comandos en la terminal (**Gemini CLI**), la nueva herramienta de comandos de Google incluye una función de migración automática. Al ejecutar por primera vez la nueva consola de Antigravity (**agy**), te aparecerá un asistente interactivo. Si prefieres forzar la importación manualmente desde tu terminal, escribe: [2, 8, 9]

```
agy plugin import gemini
```

Una vez que abras tu nuevo Antigravity IDE, verás que tu entorno, atajos de teclado y paletas de colores se habrán restaurado exactamente como los tenías. [5]

¿Pudiste localizar las carpetas originales en tu sistema o prefieres que te ayude a revisar si tus perfiles antiguos quedaron guardados bajo alguna ruta de respaldo diferente?

- [1] <https://dev.to/turingsoracle/antigravity-is-dead-long-live-antigravity-186r>
- [2] <https://www.youtube.com/watch?v=dWD7Unllcdk&t=7>
- [3] https://www.reddit.com/r/google_antigravity/comments/1tih8bf/how_to_migrate_your_settings_and_extensions_from/?tl=es-es
- [4] <https://www.youtube.com/watch?v=fXj7lG4V1IU>
- [5] https://www.reddit.com/r/google_antigravity/comments/1tih8bf/

how_to_migrate_your_settings_and_extensions_from/

[6] <https://discuss.ai.google.dev/t/antigravity-ide-2-0-conversation-history-migration-165643>

[7] https://www.reddit.com/r/GeminiAI/comments/1ti01zq/updated_antigravity_now_its_version_20_and_all_my/

[8] <https://antigravity.google/docs/gcli-migration?authuser=19&hl=es>

[9] <https://antigravity.google/docs/gcli-migration?authuser=01&hl=it&hl=it>

entonces si las funciones de “Gemini Code Assist” vienen preinstaladas de forma nativa como la IA principal en la versión 2.0, ¿significa que ahora “Gemini Code Assist” es la LLM orquestadora de los demás agentes de IA en esa versión?

No exactamente, no hay que confundir la marca comercial con el modelo de lenguaje de orquestación.

“Gemini Code Assist” es el nombre comercial del producto enfocado en desarrollo (anteriormente conocido como Duet AI), pero **el LLM que actúa como el verdadero cerebro u orquestador en Google Antigravity 2.0 es el modelo fundacional Gemini 3 Pro**. [1, 2, 3]

La arquitectura multi-agente de la versión 2.0 funciona bajo una lógica descentralizada y abierta muy particular: [1, 4]

- **El Gestor de Agentes (Agent Manager):** Antigravity 2.0 incluye un panel central nativo que se encarga de recibir tu instrucción en lenguaje natural, trazar el plan de ejecución y delegar las tareas secundarias a distintos subagentes en paralelo (como escribir pruebas, refactorizar componentes o revisar bases de datos). [3, 4, 5]
- **Soporte multi-modelo nativo:** Aunque **Gemini 3 Pro** viene configurado por defecto para la orquestación general y razonamientos complejos, la plataforma de Google es agnóstica. Permite cambiar el modelo asignado a cada subagente individual. Puedes configurar la plataforma para que un subagente use modelos externos de la competencia, como **Claude 3.5 Sonnet** o **GPT-OSS**, si consideras que rinden mejor para una tarea de código o automatización específica. [1, 3]
- **Integración profunda (Deep Integration):** La razón por la que “Gemini Code Assist” deja de existir como extensión independiente para fusionarse en este entorno es de rendimiento. Al estar embebido en el núcleo de Antigravity, el modelo de Google tiene acceso directo a las herramientas del sistema (como la consola **agy** o la API de agentes

administrados), eliminando la latencia que causaban las extensiones de los IDE tradicionales. [2, 4, 6]

En conclusión: Gemini Code Assist fue absorbido por la plataforma. Ahora el flujo operativo es controlado por la infraestructura agéntica de **Antigravity 2.0**, la cual utiliza por defecto a **Gemini 3 Pro** como el director de la orquesta, pero con la libertad de delegar tareas a otros LLMs del mercado. [1, 3, 6, 7]

¿Te interesa ver un ejemplo de cómo **configurar un flujo de trabajo asignando diferentes modelos** (como Gemini y Claude) a subagentes específicos en un mismo proyecto?

- [1] <https://antigravityide.net/>
 - [2] <https://www.facebook.com/groups/882824057510049/posts/954218137037307/>
 - [3] https://www.reddit.com/r/ChatGPTCoding/comments/1p35bdl/i_tried_googles_new_antigravity_ide_so_you_dont/
 - [4] <https://www.buildmvpfast.com/blog/google-antigravity-2-framework-cli-agent-de>
 - [5] <https://www.youtube.com/watch?v=PV3iPRVADxU&t=4>
 - [6] <https://www.instagram.com/p/DY2VxIhqcxB/>
 - [7] <https://antigravity.google/product/antigravity-2>
- si por favor.

Para configurar un flujo de trabajo multi-modelo en **Google Antigravity 2.0**, se utiliza el panel de **Customizations** (Personalizaciones) o se estructuran archivos Markdown especiales llamados **Agent Skills**. Estos archivos sirven como “módulos programables” que dictan a qué IA delegar cada subtarea. [1, 2, 3, 4]

Para lograr que **Gemini** organice y **Claude** ejecute código de forma combinada, sigue este proceso paso a paso:

1. Crear el flujo de trabajo (Workflow)

1. Abre **Antigravity 2.0** y ve al **Agent Manager** (Administrador de agentes).
2. En la esquina superior derecha, haz clic en el menú de tres puntos (...) y selecciona **Customizations**.
3. Ve a la pestaña **Workflows** y presiona + **Workspace** para aplicarlo solo a tu proyecto actual.
4. Nombra el flujo como `desarrollo_mixto`. [2, 3, 5]

2. Configurar la asignación de LLMs en el Content

Dentro del cuadro de texto del contenido (Content), pega la estructura de orquestación. Antigraity lee las instrucciones en Markdown e interpreta las etiquetas de enrutamiento de modelos de forma nativa. Copia este ejemplo: [2, 3]

```
# Workflow: Desarrollo y Pruebas Mixtas
```

```
Description: Orquesta tareas dividiendo el trabajo entre Gemini y Claude.
```

```
## Step 1: Planificación y Arquitectura
```

```
- Agent: "Architect_Agent"
```

```
- Primary_Model: "gemini-3-pro"
```

```
- Instruction: >
```

```
    Analiza el repositorio actual. Diseña la estructura técnica para la nueva funcionalidad solicitada por el usuario y genera la lista de tareas (Artifact).
```

```
## Step 2: Escritura de Código
```

```
- Agent: "Coder_Agent"
```

```
- Primary_Model: "claude-sonnet-4.5"
```

```
- Instruction: >
```

```
    Toma el Artifact generado por el Architect_Agent en el paso 1. Escribe el código fuente correspondiente respetando las directrices arquitectónicas. Abre la terminal interna si necesitas instalar paquetes.
```

```
## Step 3: Generación de Pruebas Unitarias
```

```
- Agent: "QA_Agent"
```

```
- Primary_Model: "gemini-3-flash"
```

```
- Instruction: >
```

```
    Lee el código generado por Coder_Agent. Genera la suite de pruebas unitarias con cobertura del 90%. Ejecuta los tests en el entorno aislado para verificación.
```

3. Ejecutar el flujo multi-modelo

Una vez que guardes el archivo, la plataforma compilará la receta. Para activarla dentro del chat de tu proyecto, simplemente invoca el comando con una barra diagonal: [2, 3]

```
/desarrollo_mixto "Crea un sistema de autenticación JWT para la API"
```


¿Qué pasará en tu pantalla al ejecutarlo?

El **Agent Manager** (Mission Control) iniciará la tarea usando **Gemini 3 Pro** como cerebro base. En la barra lateral verás cómo se abren dinámicamente los subagentes: [5, 6, 7]

- Verás un indicador verde que dice [Architect_Agent - Gemini 3 Pro] analizando tus archivos.
- Automáticamente, la tarea pasará al segundo subagente, cambiando el indicador a [Coder_Agent - Claude 4.5] mientras se redacta el código fuente de forma masiva.
- Por último, **Gemini 3 Flash** entrará en acción para programar los tests de forma rápida y económica. [6, 8]

Al finalizar el flujo, la app te presentará un **Artifact** (un entregable listo) con la lista de archivos modificados para que le des tu aprobación final antes de guardarlos. [5, 9]

¿Te gustaría que diseñemos un flujo específico para tu lenguaje de programación actual o prefieres ver cómo conectar bases de datos utilizando las **Agent Skills** de Antigravity? [1, 10]

- [1] <https://antigravity.google/docs/skills>
- [2] <https://antigravity.google/docs/rules-workflows>
- [3] <https://agentpedia.codes/blog/workflows>
- [4] <https://www.youtube.com/watch?v=yuaBPLNdNSU&v1=es>
- [5] <https://www.youtube.com/watch?v=5vSKhKsvayA>
- [6] <https://www.youtube.com/watch?v=sEk6fcdSYmI&t=20>
- [7] <https://discuss.ai.google.dev/t/multi-agent-use-in-antigravity/117941>

- [8] <https://www.eigent.ai/blog/antigravity-vs-claude-cowork>
- [9] <https://developers.googleblog.com/build-with-google-antigravity-our-new-agent>
- [10] <https://codelabs.developers.google.com/getting-started-with-antigravity-skill>

este flujo de trabajo multimodelo permite ahorrar tokens?

Sí, y de forma drástica. El diseño de flujos de trabajo multi-modelo en Google Antigravity 2.0 es, actualmente, una de las mejores estrategias para **reducir el consumo de tokens y optimizar los costos operativos**.

Si utilizaras un solo modelo de gama alta (como *Gemini 3 Pro* o *Claude Sonnet*) para absolutamente todo el proceso, gastarías una fortuna en tokens de contexto. Al dividir las tareas, ahorras tokens mediante tres mecanismos clave:

1. Asignación por complejidad (Modelo Inteligente vs. Modelo Económico)

No todas las tareas requieren el mismo nivel de razonamiento. El flujo te permite pagar el “precio premium” de los tokens solo cuando es estrictamente necesario:

- **Planificación (Cerebro):** Usas *Gemini 3 Pro* solo para el paso 1 (analizar la arquitectura). Pagas tokens caros solo por la estrategia.
- **Escritura (Ejecutor):** Pasas la tarea a un modelo especializado en código.
- **Pruebas (Editor/Corrector):** La generación de pruebas unitarias o documentación es repetitiva. Al delegarla a **Gemini 3 Flash** en el paso 3, procesas miles de líneas de código usando un modelo que cuesta hasta **10 veces menos por token** que las versiones Pro.

2. Segmentación del Contexto (Menos “Basura” en la Memoria)

En un chat tradicional de IA, cada vez que envías un mensaje nuevo, el sistema vuelve a leer **toda la conversación anterior** (efecto bola de nieve de tokens). Antigravity 2.0 rompe este ciclo:

- El **QA Agent** (Gemini Flash) **solo recibe el código generado en el paso anterior**.
- No necesita leer las discusiones arquitectónicas, los intentos fallidos ni el historial completo del chat.
- Al aislar el contexto de cada subagente, el consumo de tokens de entrada (*input tokens*) se mantiene lineal en lugar de exponencial.

3. Uso Eficiente de los “Artifacts” (Entregables)

En lugar de pasarse archivos enteros y pesados entre agentes, el sistema utiliza los **Artifacts**. Un agente condensa la información en un resumen técnico estructurado (un artefacto de pocas líneas) y se lo pasa al siguiente.

Esto evita inyectar miles de líneas de código fuente redundante en la ventana de contexto del próximo modelo.

Estimación de Ahorro Real

En proyectos medianos o grandes, implementar esta división de tareas (Pro para estructurar → Especializado para programar → Flash para testear y documentar) suele generar un **ahorro de entre el 40 % y el 60 % en tu factura de tokens** en comparación con hacer todo el desarrollo dentro de un chat lineal con un único modelo avanzado.

¿Te gustaría que optimicemos el flujo anterior añadiendo **instrucciones de compresión** en los Artifacts para reducir aún más el consumo de tokens de entrada?

si por favor.

El truco definitivo para congelar el gasto en tu factura de tokens dentro de Google Antigravity 2.0 consiste en indicarle explícitamente a los agentes que dejen de transmitirse archivos de código completos de un paso al otro. En su lugar, se les obliga a comunicarse usando **resúmenes de contexto comprimidos (Prompt Compression)**. [1, 2, 3]

Aquí tienes el flujo de trabajo optimizado. Copia y reemplaza esta receta en la pestaña **Workflows** de tu panel de personalizaciones: [4]

Workflow: Desarrollo Mixto con Ultra-Ahorro de Tokens

Description: Orquestación multi-modelo con reglas estrictas de compresión de con

Step 1: Planificación y Arquitectura Minimalista

- Agent: "Architect_Agent"

- Primary_Model: "gemini-3-pro"

- Instruction: >

Analiza el repositorio actual. Diseña la solución técnica para el requerimie

REGLA DE COMPRESIÓN DE TOKENS: No generes explicaciones largas ni código fue

Entrega el plan de forma comprimida exclusivamente en formato 'Pseudo-código

Este entregable (Artifact) debe pesar menos de 400 tokens obligatoriamente.

Step 2: Escritura de Código Focalizada

- Agent: "Coder_Agent"

- Primary_Model: "claude-sonnet-4.5"

- Instruction: >

Lee únicamente el Artifact JSON compacto generado en el paso 1.

Escribe el código fuente correspondiente para cumplir con el JSON.
REGLA DE COMPRESIÓN DE TOKENS: No devuelvas explicaciones ni repitas código.
Genera un Artifact de salida que contenga estrictamente el bloque de código.
Omite los nombres de las funciones modificadas. Omite comentarios redundantes.

Step 3: Pruebas Unitarias de Bajo Costo

- Agent: "QA_Agent"

- Primary_Model: "gemini-3.5-flash"

- Instruction: >

Toma el bloque de código nuevo generado por Coder_Agent en el paso 2.

No leas el historial de conversación anterior para evitar el desperdicio de tokens.

Genera la suite de pruebas unitarias enfocándote únicamente en el nuevo bloque de código.

Ejecuta el comando en el entorno aislado para validar el resultado.

¿Por qué este flujo exprime al máximo tus tokens?

1. **La técnica del “Pseudo-código JSON”:** Obligar a *Gemini 3 Pro* a dar una respuesta estructurada en JSON en el Paso 1 reduce el volumen de texto instructivo. Al no incluir introducciones, párrafos explicativos ni saludos cordiales, el artefacto resultante consume apenas una fracción de los tokens de entrada (*input tokens*) en el Paso 2. [3, 5]
2. **Uso de Gemini 3.5 Flash:** Google optimizó drásticamente el costo de **Gemini 3.5 Flash** en la versión 2.0. Al derivar la generación de pruebas (el paso que más volumen de líneas de texto plano genera) a este modelo en el Paso 3, realizas el trabajo pesado por un costo monetario ínfimo en comparación con los modelos de frontera. [1, 6, 7]
3. **Aislamiento de la Historia:** La orden dada al agente del Paso 3 de “*No leas el historial de conversación anterior*” instruye al backend de Antigravity a realizar un truncamiento de la memoria interna. Esto detiene por completo el efecto “bola de nieve”, evitando que pagues una y otra vez por los mismos tokens acumulados en el chat durante los turnos previos. [2, 5, 7]

Con esta estructura de compresión manual e inyección de **Gemini 3.5 Flash**, el consumo de tokens de tu entorno se reducirá notablemente. [1, 5, 7]

Si utilizas librerías externas o frameworks específicos, puedo enseñarte cómo **inyectar esquemas ligeros (schemas compactos) de tus librerías** para que el agente escriba código preciso sin tener que leer toda la documentación del framework. ¿Te gustaría ver un ejemplo? [5]

- [1] <https://www.youtube.com/watch?v=RZ1ZSRuShn4>
 - [2] <https://thenewstack.io/how-to-reduce-mcp-token-bloat/>
 - [3] <https://www.ibm.com/think/tutorials/prompt-compression>
 - [4] https://www.reddit.com/r/google_antigravity/comments/1thxv5g/introducing_google_antigravity_20/?tl=es-419
 - [5] <https://www.mindstudio.ai/blog/optimize-mcp-server-token-usage>
 - [6] <https://www.youtube.com/watch?v=PV3iPRVADxU>
 - [7] <https://www.mindstudio.ai/blog/ai-agent-token-cost-optimization-multi-model-r>
- si por favor me gustaría ver un ejemplo.

Para inyectar esquemas ligeros (*schemas*) de librerías y evitar que los agentes lean gigabytes de documentación consumiendo tus tokens, se utiliza una característica nativa de **Google Antigravity 2.0** llamada **Knowledge Mappings** (o Esquemas de Referencia Rápida). [1, 2]

El truco consiste en crear un archivo JSON o Markdown ultracompacto que resuma los métodos, firmas y tipos exactos de la librería que vas a usar. El agente leerá este mapa en milisegundos en lugar de ir a buscar el código fuente de `node_modules` o paquetes de Python. [3]

A continuación, tienes un ejemplo práctico de cómo estructurar e inyectar el esquema si estuvieras desarrollando una API con **FastAPI** (Python) o **Prisma ORM**:

1. El Archivo de Esquema Compacto (`.antigravity/schemas`)

Crea un archivo oculto en la raíz de tu proyecto o en tu carpeta de configuración global que contenga únicamente las firmas y el formato exacto que requieres, eliminando explicaciones textuales:

```
{
  "library": "FastAPI + Prisma Auth Stub",
  "version_target": "0.110+",
  "allowed_routing_patterns": {
    "APIRouter": "from fastapi import APIRouter; router = APIRouter(prefix='/v1/'",
    "Response_Format": "return {'status': 'success', 'data': dict}"
  },
  "prisma_model_stubs": {
    "User": {
```

```

        "query_methods": [
            "db.user.find_unique(where={'email': str})",
            "db.user.create(data={'email': str, 'password_hash': str, 'role': 'USER'})",
        ],
        "returns": "prisma.models.User object or None"
    }
},
"security_dependencies": {
    "JWT_Bearer": "from app.core.security import get_current_user; Depends(get_current_user)"
}
}

```

2. El Workflow Multi-Modelo con Inyección Explícita

Ahora, en tu panel de **Customizations > Workflows** de Antigravity, configuras al agente ejecutor para que use ese archivo JSON como su única verdad absoluta para programar. [2, 4]

Al apuntar el contexto directamente a este archivo pequeño, bloqueas que el agente intente indexar todo el SDK del framework: [4]

```

# Workflow: Generación de API de Alta Precisión
Description: Escribe endpoints de autenticación inyectando esquemas locales para

## Step 1: Programación Asistida por Esquema
- Agent: "Backend_Coder_Agent"
- Primary_Model: "claude-sonnet-4.5"
- Supporting_Context: "./.antigravity/schemas/fastapi-prisma.json"
- Instruction: >
    Debes escribir el código del archivo 'src/routes/auth.py'.
    REGLA DE CONTEXTO ESTRICTA: No inventes métodos de FastAPI ni de Prisma.
    Usa única y exclusivamente las firmas de código declaradas en el archivo
    inyectado en 'Supporting_Context'.
    Si necesitas una función que no esté en el JSON, pregunta al usuario en lugar
    de adivinar o buscar en la web.
    Entrega solo el bloque de código limpio en un Artifact.

## Step 2: Validación Estática
- Agent: "Linter_Agent"
- Primary_Model: "gemini-3.5-flash"

```

- Instruction: >

Revisa el código generado en el paso anterior.

Verifica que las llamadas a 'db.user' coincidan exactamente con la estructura del esquema JSON inyectado previamente. Si hay discrepancias, corrige el archivo.

El Impacto Económico en tus Tokens

- **Sin Esquema Inyectado:** Cuando un agente lee código donde importas `from fastapi import FastAPI`, el sistema de autocompletado tradicional o el agente de Antigravity intenta indexar los archivos internos de la librería para entender los métodos. Esto puede inflar tu contexto inicial sumando entre **15,000 y 50,000 tokens innecesarios** de trasfondo.
- **Con Esquema Inyectado:** Tu archivo JSON de referencia pesa escasos **250 tokens**. El agente tiene las instrucciones precisas en la memoria de trabajo a corto plazo, el código no sufre de alucinaciones sintácticas y la velocidad de respuesta del modelo se duplica al no tener que procesar dependencias gigantescas. [3, 5]

¿Qué frameworks o librerías específicas estás utilizando en tu proyecto actual? Podríamos **diseñar juntos el archivo de esquema JSON exacto** para tu entorno de desarrollo.

- [1] <https://ecosistemastartup.com/google-antigravity-2-0-crea-agentes-ia-con-1>
- [2] https://www.youtube.com/watch?v=7jw0hZQ__Pk
- [3] <https://www.youtube.com/watch?v=MhJsmN1Mm0w>
- [4] <https://antigravity.google/>
- [5] <https://www.youtube.com/watch?v=3l0SRwtBAnM>